Topic 1: Understand Behavior as Data (Beginner Level)

This is one of the biggest ideas introduced in **Java 8**. Once you understand it, you'll easily learn **Lambda Expressions, Functional Interfaces, Method References, Streams, Optional, and the Stream API**.

Before Java 8

Until Java 7, Java mostly treated **data** as something that could be stored in variables.

```
int age = 22;
String name = "Siraj";
double salary = 50000;
```

Here,

- 22 is data
- "Siraj" is data
- 50000 is data

But **methods (behavior)** could **not** be stored inside variables.

```
public void greet() {
    System.out.println("Hello");
}
```

You could call it

```
greet();
```

But you couldn't do something like

```
behavior = greet;   // Not possible before Java 8
```

What is Behavior?

Behavior simply means

> **What an object does.**

Example

A Dog

Data

```
name = Bruno
age = 4
```

Behavior

```
bark()
eat()
sleep()
```

Data describes the object.

Behavior performs an action.

What does "Behavior as Data" mean?

Instead of passing only values,

Java 8 allows us to pass **behavior (code)** as an argument.

Think of it as

Instead of saying

> "Here's the number."

You can now say

> "Here's the work you should perform."

Example

Old way

```
calculate(5);
```

Java 8 way

```
calculate(() -> System.out.println("Hello"));
```

Instead of giving data,

you're giving **a piece of behavior**.

Real Life Analogy

Imagine ordering food.

Old Java

You tell the chef

```
Cook Biryani.
```

New Java

You hand over the recipe.

```
Whenever you're ready,
follow these instructions.
```

The recipe is **behavior**.

Why Java Needed This?

Suppose you want to sort employees.

Different situations require different sorting.

Sort by

- name
- age
- salary
- experience

Old Java

You had to create many classes.

```
AgeComparator

SalaryComparator

NameComparator

ExperienceComparator
```

Java 8

You simply pass the behavior.

```
employees.sort((a, b) -> a.getAge() - b.getAge());
```

The sorting logic is treated like data.

Another Example

Suppose

```
void executeTask(Runnable task)
```

You can pass

```
executeTask(() -> System.out.println("Running"));
```

You're passing

```
"what to do"
```

instead of

```
"some value"
```

Before Java 8

To pass behavior, you needed anonymous classes.

```
Runnable task = new Runnable() {
    @Override
    public void run() {
        System.out.println("Running");
    }
};
```

Now

```
Runnable task = () -> System.out.println("Running");
```

Exactly the same behavior.

Much smaller code.

Why is this Powerful?

Because Java libraries can now accept

```
behavior
```

instead of only

```
objects
```

Example

Filtering

```
list.removeIf(x -> x < 10);
```

The method doesn't know the condition.

You provide it.

Sorting

```
list.sort((a, b) -> a.compareTo(b));
```

Again,

the sorting logic comes from you.

Streams

```
list.stream()
    .filter(x -> x > 5)
```

```
    .map(x -> x * 2)
    .forEach(System.out::println);
```

Each lambda is behavior.

Connection with Upcoming Topics

Behavior as Data is the foundation of:

```
Functional Interface
        ↓
Lambda Expression
        ↓
Method Reference
        ↓
Streams
        ↓
Optional
        ↓
Functional Programming
```

If this concept isn't clear, all of these topics become difficult.

Common Beginner Mistake

Thinking

> Lambda is behavior.

Not exactly.

Lambda is **a way to represent behavior**.

Behavior is the concept.

Lambda is the syntax.

Real Project Usage

Spring Boot uses behavior as data everywhere.

Examples

```
Streams

CompletableFuture

ExecutorService

Runnable

Callable

Event Listeners

Comparators

Predicates

Consumers

Suppliers

Spring Security Filters
```

Interview Questions

Q1. What does "Behavior as Data" mean?

Answer

Passing executable code as an object so that another method can execute it later.

Q2. Which Java version introduced this concept?

Java 8.

Q3. What made Behavior as Data possible?

Functional Interfaces + Lambda Expressions.

Q4. Can Java pass methods directly?

No.

Java passes an object implementing a Functional Interface.

Q5. Is a lambda an object?

Yes.

At runtime, the JVM creates an object implementing the target Functional Interface.

Tricky Interview Questions

Q

Can Java store methods inside variables?

Answer

Not directly.

It stores an object representing the method (typically via a Functional Interface).

Q

Is behavior itself data?

Not literally.

Java wraps behavior inside an object, allowing it to be passed around like data.

Q

Can every method become behavior as data?

No.

Only methods compatible with a Functional Interface's single abstract method signature can be used.

Interview Summary

Remember these keywords

- Java 8 feature
- Functional Programming
- Pass behavior
- Lambda
- Functional Interface
- Anonymous Class replacement
- Deferred execution
- Strategy Pattern simplified

One-Liner Interview Answer

> **Behavior as Data means passing executable logic (behavior) through Functional Interface objects, allowing methods to receive and execute code just like they receive ordinary data.**

Topic 2: Create a Custom Functional Interface with One Abstract Method

A **Functional Interface** is an interface that contains **exactly one abstract method**.

It acts as a contract for behavior and is the foundation of lambda expressions.

Why Do We Need Functional Interfaces?

Suppose you want different greeting behaviors:

- Say Hello

- Say Good Morning
- Say Welcome

Instead of creating many classes, define one behavior contract.

```
@FunctionalInterface
interface Greeting {
    void greet();
}
```

The single abstract method greet() defines the behavior.

Implementing the Interface (Traditional Way)

```
class HelloGreeting implements Greeting {

    @Override
    public void greet() {
        System.out.println("Hello!");
    }
}
```

Usage:

```
Greeting g = new HelloGreeting();
g.greet();
```

Output:

```
Hello!
```

Implementing with a Lambda (Java 8+)

```
Greeting g = () -> System.out.println("Hello!");

g.greet();
```

Notice there is **no anonymous class**. The lambda provides the implementation of the single abstract method.

Functional Interface with Parameters

```
@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}
```

Using a lambda:

```
Calculator calc = (a, b) -> a + b;

System.out.println(calc.add(10, 20));
```

Output:

```
30
```

Returning a Value

```
@FunctionalInterface
interface Square {
```

```
    int square(int n);
}

Square s = n -> n * n;

System.out.println(s.square(6));
```

Output:

```
36
```

Can a Functional Interface Have Other Methods?

Yes.

It can have:

- One abstract method ✅
- Any number of default methods ✅
- Any number of static methods ✅
- Methods inherited from Object (like toString()) ✅

Example:

```
@FunctionalInterface
interface Printer {

    void print(String msg);

    default void start() {
        System.out.println("Starting...");
    }

    static void stop() {
        System.out.println("Stopped");
```

```
    }
}
```

Still a valid Functional Interface because there is only **one abstract method**.

Why Use @FunctionalInterface?

```
@FunctionalInterface
interface Test {
    void show();
}
```

This annotation is optional but recommended.

If you accidentally add another abstract method:

```
@FunctionalInterface
interface Test {

    void show();

    void display();   // Compile-time error
}
```

The compiler immediately reports an error, preventing accidental misuse.

How It Connects to Behavior as Data

Behavior:

```
(a, b) -> a + b
```

needs a target type.

The Functional Interface provides that target.

```
Behavior (Lambda)
        ↓
Implements
        ↓
Functional Interface
        ↓
Passed to methods
```

Without a Functional Interface, Java wouldn't know what method the lambda is implementing.

Real Project Usage

You'll frequently encounter built-in functional interfaces:

- Runnable
- Callable
- Comparator
- Predicate
- Function
- Consumer
- Supplier
- BiFunction

Custom Functional Interfaces are useful when your project needs behavior that isn't covered by the standard ones.

Common Mistakes

❌ Multiple abstract methods

```
interface Demo {
    void a();
    void b();
}
```

Not a Functional Interface.

❌ Forgetting that default methods are allowed

```
default void log() { }
```

This is perfectly valid.

❌ Thinking @FunctionalInterface creates functionality

It doesn't.

It only asks the compiler to verify that the interface has exactly one abstract method.

Interview Questions

Q1. What is a Functional Interface?

An interface with exactly one abstract method that can be implemented using a lambda expression.

Q2. Is @FunctionalInterface mandatory?

No. It is optional but strongly recommended for compile-time validation.

Q3. Can it contain default methods?

Yes.

Q4. Can it contain static methods?

Yes.

Q5. Can it extend another interface?

Yes, provided the resulting interface still has only one abstract method.

Q6. Why are Functional Interfaces important?

Because lambda expressions require a target Functional Interface.

Tricky Interview Questions

Q

Can a Functional Interface have 10 default methods?

Answer: Yes. The limit applies only to **abstract** methods.

Q

Does equals() count as an abstract method?

Answer: No. Methods inherited from Object don't count.

Q

Can an interface without @FunctionalInterface still be a Functional Interface?

Answer: Yes, as long as it has exactly one abstract method.

Q

Can you instantiate a Functional Interface directly?

Answer: No. You instantiate an implementing object, often using a lambda or an anonymous class.

Interview Summary

Remember these keywords:

- Exactly one abstract method
- Target type for lambdas
- Java 8
- @FunctionalInterface
- Default methods allowed

- Static methods allowed
- Object methods ignored
- Basis of Stream API and functional programming

One-Liner Interview Answer

> **A Functional Interface is an interface with exactly one abstract method, serving as the target type for lambda expressions and enabling behavior to be passed as data in Java.**